

KUBERNETES

50 Scenario-Based Interview Questions

Deep-Dive Answers for 2–5 Years Experience

Covers: Pods • Networking • Storage • RBAC • Scaling • StatefulSets • Cluster Ops • Security • Advanced Patterns

March 2026 · 10 Categories · 50 Questions · Production-Level Answers

01 Pods & Workloads

Q01 Pod Troubleshooting

A Pod is stuck in Pending state for 10 minutes. Walk me through how you diagnose and resolve it.

Answer:

A Pod stuck in Pending means the Scheduler has NOT yet assigned it to any node. The root cause is almost always one of three things: insufficient resources, a scheduling constraint mismatch, or a missing PVC.

Step 1 — Describe the Pod: Run `kubectl describe pod <name>`. Focus on the Events section at the bottom. Look for FailedScheduling events.

Step 2 — Check resources: If the event says Insufficient cpu or Insufficient memory, the cluster has no node with enough headroom. Either scale the cluster, reduce resource requests, or move the Pod.

Step 3 — Check taints/tolerations: If the event says 0/3 nodes are available: 3 node(s) had intolerated taints, add the correct toleration to the Pod spec.

Step 4 — Check node selectors/affinity: If nodeSelector or nodeAffinity is defined, verify matching nodes exist with `kubectl get nodes --show-labels`.

Step 5 — Check PVC: If the Pod uses a PersistentVolumeClaim, run `kubectl get pvc`. A Pending PVC blocks the Pod from starting.

```
kubectl describe pod <pod-name>           # Read Events section
kubectl get nodes -o wide                  # Node resource overview
kubectl describe nodes | grep -A5 Taints
kubectl get pvc -n <namespace>            # Check volume binding
```

Key Takeaway: Pending = scheduling problem, not application problem | Events section in describe is always step 1

Q02 CrashLoopBackOff

Your application Pod is in CrashLoopBackOff. What are your steps, and what are the most common root causes?

Answer:

CrashLoopBackOff means the container starts, crashes quickly, and Kubernetes keeps retrying with exponential backoff (10s → 20s → 40s → 80s → 160s → 300s max). The container IS running; the problem is inside the app.

Step 1 — Read logs: `kubectl logs <pod> --previous` (--previous fetches logs from the crashed instance, not the current one).

Step 2 — Check exit code: `kubectl describe pod` shows the exit code. Exit 1 = application error, Exit 137 = OOMKilled (out of memory), Exit 139 = segfault.

Step 3 — Common causes:

- Wrong environment variable (missing DB_HOST, bad credentials)
- Application fails to connect to a dependency (DB, Redis, external API)
- OOMKilled — memory limit too low, raise `limits.memory`
- Wrong image entrypoint or command in the Pod spec
- Missing ConfigMap or Secret that is mounted as a volume

```
kubectl logs <pod> --previous          # Logs from crashed instance
kubectl describe pod <pod> | grep -A3 'Last State'
kubectl describe pod <pod> | grep 'Exit Code'
kubectl exec -it <pod> -- /bin/sh      # Debug if pod stays up long enough
```

Key Takeaway: `--previous` flag is crucial | Exit 137 = OOMKilled → increase memory limit

Q03 Rolling Update Stuck

You performed a `kubectl rollout` and it is stuck with the new Pods not becoming Ready. How do you handle it?

Answer:

A stuck rollout means the new ReplicaSet's Pods are failing health checks or the readinessProbe is not passing. The old Pods remain running because Kubernetes waits before terminating them.

Check rollout status: `kubectl rollout status deployment/<name>` will tell you how many replicas are updated/available.

Inspect new Pods: `kubectl get pods` shows the new Pods (look for the new hash suffix). Describe them to find probe failures.

Options:

- Fix the application issue and push a new image version
- Rollback immediately: `kubectl rollout undo deployment/<name>`
- Pin to a specific revision: `kubectl rollout undo deployment/<name> --to-revision=2`

The deployment's `maxUnavailable` and `maxSurge` settings control how aggressive the rollout is. Setting `maxUnavailable: 0` ensures zero downtime but requires sufficient cluster capacity.

```
kubectl rollout status deployment/my-app
kubectl rollout history deployment/my-app
kubectl rollout undo deployment/my-app
kubectl rollout undo deployment/my-app --to-revision=2
```

Key Takeaway: Rollback is instant — old ReplicaSet is never deleted | `maxUnavailable:0` + `maxSurge:1` = zero-downtime rollout

Q04 Init Container Failing

A Pod's main container never starts because an init container is stuck. How do you debug it?

Answer:

Init containers run sequentially before the main container. If any init container fails or hangs, the entire Pod stays in Init:0/N status indefinitely.

Check init container logs: Use `kubectl logs <pod> -c <init-container-name>`. The `-c` flag selects the specific container.

Check status: `kubectl describe pod` shows each init container status separately with its own exit code.

Common scenarios: An init container waits for a service (e.g., `nc -z postgres 5432`) but the service doesn't exist or has a wrong name. Another common case: the init container image cannot be pulled (`ImagePullBackOff`).

Resolution: Fix the underlying issue (deploy the missing service, fix DNS name, fix image tag) — the Pod will automatically retry once resources become available.

```
kubectl get pod <pod> -o jsonpath='{.status.initContainerStatuses}'
kubectl logs <pod> -c init-wait-for-db
kubectl describe pod <pod> | grep -A10 'Init Containers'
```

Key Takeaway: Use `-c` flag for logs of specific containers | Init failures restart the entire Pod, not just the init container

Q05 OOMKilled Repeatedly

Your Pod is repeatedly OOMKilled. What do you do to investigate and permanently fix it?

Answer:

OOMKilled (exit code 137) means the container exceeded its memory limit and the Linux kernel killed it.

Kubernetes does not restart it — the kubelet does, per the `restartPolicy`.

Confirm the cause: `kubectl describe pod` shows Reason: OOMKilled under Last State.

Measure actual usage: Use `kubectl top pod <name>` to see real-time memory consumption. Compare with the configured `limits.memory`.

Fix options:

- Increase `limits.memory` in the Pod spec — the fastest fix
- Investigate memory leaks in the application with profiling tools
- Add Vertical Pod Autoscaler (VPA) to automatically right-size the container
- Use a memory-efficient image (Alpine-based, JVM heap tuning for Java)

For Java workloads specifically, set `-Xmx` to ~75% of `limits.memory` to leave headroom for JVM overhead, GC, and native threads.

```
kubectl describe pod <pod> | grep -A5 'Last State'
kubectl top pod <pod> --containers
# Fix in deployment spec:
resources:
```

```
limits:
  memory: 512Mi    # Increase this
requests:
  memory: 256Mi
```

Key Takeaway: Exit 137 = OOM | kubectl top needs metrics-server installed | VPA automates right-sizing

02 Networking & Services

Q06 Service Not Reachable

A Service is created but Pods cannot reach it by its DNS name. How do you troubleshoot?

Answer:

Service DNS resolution in Kubernetes follows the pattern: <service>.<namespace>.svc.cluster.local. When it breaks, the issue is usually DNS, label selector mismatch, or wrong port.

Step 1 — Verify selector matches Pod labels: kubectl describe service <name> shows the selector. kubectl get pods --show-labels verifies the Pod has those exact labels.

Step 2 — Check Endpoints: kubectl get endpoints <service>. If endpoints shows <none>, the selector is wrong or no Pods are Ready.

Step 3 — Test DNS: Exec into a Pod and run nslookup <service>.<namespace>.svc.cluster.local. If DNS fails, CoreDNS is the problem.

Step 4 — Check CoreDNS: kubectl get pods -n kube-system | grep coredns — ensure CoreDNS Pods are Running and Ready.

Step 5 — Port mismatch: The Service port and the Pod containerPort must match. Also confirm the targetPort matches the actual port the app listens on.

```
kubectl describe svc my-service          # Check selector
kubectl get endpoints my-service         # Must not be <none>
kubectl get pods --show-labels          # Verify label match
kubectl exec -it debug-pod -- nslookup my-service.default.svc.cluster.local
kubectl logs -n kube-system -l k8s-app=kube-dns
```

Key Takeaway: Endpoints = <none> means selector mismatch | Always verify label keys AND values exactly

Q07 LoadBalancer Pending

A Service of type LoadBalancer stays in Pending state for its external IP. What are the possible causes?

Answer:

LoadBalancer type Services require a cloud provider integration (AWS ELB, GCP LB, Azure LB) to provision an external IP. If the cluster lacks this integration, it stays Pending forever.

- On-premises / bare metal: LoadBalancer has no provider. Use MetalLB or change to NodePort.

- Cloud cluster: Check if the cloud-controller-manager is running and has IAM permissions to create load balancers.
- Resource quota: The cloud account may have hit its load balancer quota.
- Wrong annotation: Some clouds require specific annotations (e.g., service.beta.kubernetes.io/aws-load-balancer-type).

For development/on-prem environments, use `kubectl port-forward` for testing or deploy MetalLB which provides a software LoadBalancer IP pool.

```
kubectl describe svc my-lb-service          # Look for Events
kubectl get pods -n kube-system | grep cloud-controller
# For on-prem: install MetalLB or use NodePort
# Change type:
kubectl patch svc my-service -p '{"spec":{"type":"NodePort"}}'
```

Key Takeaway: LoadBalancer = cloud provider feature | MetalLB for bare metal | Use NodePort as fallback

Q08 Network Policy Blocking Traffic

After applying a NetworkPolicy, some inter-service communication broke. How do you debug it?

Answer:

NetworkPolicies are deny-by-default once applied to a namespace. A single NetworkPolicy selecting a Pod causes ALL ingress/egress that isn't explicitly allowed to be dropped.

Step 1 — List all policies: `kubectl get networkpolicy -n <namespace>`. A single policy applying to a Pod blocks all unspecified traffic.

Step 2 — Check which Pods are selected: Look at the podSelector in each policy. If it's {} (empty), it selects ALL Pods in the namespace.

Step 3 — Test connectivity: `kubectl exec` into the source Pod and use `curl` or `nc` to test specific ports. This confirms which paths are blocked.

Step 4 — Fix the policy: Add an explicit ingress/egress rule that allows the required traffic. Specify both the source Pod's namespace+label and the correct port.

Remember: NetworkPolicies are enforced by the CNI plugin (Calico, Cilium, Weave). If the CNI doesn't support NetworkPolicy, the policies are silently ignored.

```
kubectl get networkpolicy -n my-namespace -o yaml
# Allow ingress from specific pods:
ingress:
- from:
  - podSelector:
      matchLabels:
        app: frontend
  ports:
  - protocol: TCP
    port: 8080
```

Key Takeaway: Empty podSelector = selects ALL pods | Policies are additive — union of all matching policies | CNI must support NetworkPolicy

Q09 Ingress Not Working

An Ingress resource is created but HTTP requests return 404 or fail to reach the backend. Debug steps?

Answer:

Ingress resources require an Ingress Controller (NGINX, Traefik, AWS ALB) to be running. The Ingress object itself is just configuration — without the controller, nothing happens.

Step 1 — Verify Ingress Controller exists: `kubectl get pods -n ingress-nginx` (or your namespace). If it's not running, install it.

Step 2 — Check Ingress resource: `kubectl describe ingress <name>`. Look at the Address field (should have an IP/hostname) and the Rules section.

Step 3 — Verify backend Service: The Service and port referenced in the Ingress backend must exist. `kubectl get svc <backend-service>`.

Step 4 — Check annotations: NGINX Ingress uses specific annotations for rewrites, SSL, and timeouts. A wrong annotation can silently break routing.

Step 5 — Check IngressClass: Kubernetes 1.18+ requires `spec.ingressClassName` or the `kubernetes.io/ingress.class` annotation to match your controller.

```
kubectl get ingress -o wide          # Check Address column
kubectl describe ingress my-ingress  # Read Events
kubectl get pods -n ingress-nginx    # Controller running?
kubectl logs -n ingress-nginx <controller-pod> | tail -50
# Check IngressClass
kubectl get ingressclass
```

Key Takeaway: Ingress without a controller = no-op | IngressClass matching is required in k8s 1.18+ | Check controller logs for routing errors

Q10 DNS Resolution Slow

Applications in the cluster experience slow DNS resolution causing request timeouts. How do you address this?

Answer:

DNS slowness in Kubernetes is a well-known issue often caused by conntrack race conditions in iptables with `ndots:5` (the default), which causes 5 DNS lookups per hostname before resolving.

Root cause — ndots:5: By default, Kubernetes sets `ndots:5`. A lookup for `api.example.com` triggers 5 queries: `api.example.com.default.svc.cluster.local`, `api.example.com.svc.cluster.local`, etc. before the actual external lookup.

Fix 1 — Reduce ndots: Set `dnsConfig.options ndots: 2` in the Pod spec. External FQDNs with dots > 2 resolve directly without search domain expansion.

Fix 2 — Node-local DNS cache: Deploy NodeLocal DNSCache as a DaemonSet. It caches DNS responses on each node, drastically reducing CoreDNS load.

Fix 3 — Scale CoreDNS: Increase CoreDNS replica count if the cluster is large. `kubectl scale deployment coredns -n kube-system --replicas=4`.

```
# Pod spec DNS config:
spec:
  dnsConfig:
    options:
      - name: ndots
        value: '2'
      - name: single-request-reopen
# Check CoreDNS metrics
kubectl top pod -n kube-system -l k8s-app=kube-dns
```

Key Takeaway: `ndots:5` = 5 unnecessary lookups per query | NodeLocal DNSCache is the production solution | Append . to FQDNs to bypass search domains

03 Storage & Volumes

Q11 PVC Stuck in Pending

A PersistentVolumeClaim has been Pending for a long time. What do you check?

Answer:

A PVC stays Pending because no PersistentVolume can satisfy the claim. The matching criteria are: StorageClass name, access mode, and requested capacity.

Step 1 — Describe the PVC: `kubectl describe pvc <name>`. The Events section shows why binding failed (e.g., no matching PV found, StorageClass not found).

Step 2 — Check StorageClass: `kubectl get storageclass`. If the PVC requests a StorageClass that doesn't exist, it stays Pending forever.

Step 3 — Dynamic provisioning: If using dynamic provisioning, check that the provisioner Pod is running (e.g., EBS CSI Driver, NFS provisioner).

Step 4 — Static provisioning: If using static PVs, ensure a PV with matching storageClass, accessMode, and capacity $\geq$ PVC request exists.

Step 5 — WaitForFirstConsumer: If the StorageClass has `volumeBindingMode: WaitForFirstConsumer`, the PVC binds only when a Pod is scheduled. This is normal.

```
kubectl describe pvc my-pvc          # Read Events section
kubectl get storageclass             # Verify SC exists
kubectl get pv                      # Static PVs available?
kubectl get pods -n kube-system | grep csi # CSI driver running?
```

Key Takeaway: WaitForFirstConsumer is intentional, not a bug | StorageClass name must match exactly (case-sensitive)

Q12 Data Persistence After Pod Restart

A developer complains that data written to /data in the container disappears after a Pod restart. Fix this.

Answer:

Data written to a container's filesystem is ephemeral — it lives in the container's writable layer and is destroyed when the container restarts. To persist data, you must use a Volume.

For stateful workloads: Use a PersistentVolumeClaim (PVC). The PVC-backed volume survives Pod restarts, rescheduling, and even node failures (depending on the storage class).

For shared temp data (same Pod): Use emptyDir. It persists across container restarts within the same Pod but is deleted when the Pod is deleted.

For configuration: Use ConfigMap or Secret volumes for read-only configuration files.

For databases or stateful apps, use StatefulSets with volumeClaimTemplates. Each Pod replica gets its own PVC, which is retained even when the Pod is deleted.

```
# Add PVC volume to deployment:
spec:
  volumes:
    - name: data-vol
      persistentVolumeClaim:
        claimName: my-pvc
  containers:
    - name: app
      volumeMounts:
        - name: data-vol
          mountPath: /data
```

Key Takeaway: Container FS is ephemeral | PVC survives Pod restarts | StatefulSet for databases

Q13 StatefulSet Volume Retention

A StatefulSet was deleted and the developer wants to know if the data (PVCs) was lost. Explain retention behavior.

Answer:

By default, deleting a StatefulSet does NOT automatically delete its PVCs. This is intentional — data safety is prioritized over cleanup. The PVCs and the underlying PVs remain until manually deleted.

PVC naming convention: StatefulSet creates PVCs named <volumeClaimTemplate-name>-<statefulset-name>-<ordinal>. E.g., data-myapp-0, data-myapp-1.

Re-attaching data: If the StatefulSet is recreated with the same name, it reattaches to the existing PVCs automatically — data is preserved.

Kubernetes 1.27+ PVC retention policy: The StatefulSet spec now supports persistentVolumeClaimRetentionPolicy with WhenDeleted: Delete to auto-delete PVCs when the StatefulSet is deleted.

Manual cleanup: `kubectl delete pvc -l app=myapp` — always confirm before running this in production.

```
kubectl get pvc -l app=myapp          # PVCs still exist
# Kubernetes 1.27+ retention policy:
spec:
  persistentVolumeClaimRetentionPolicy:
    whenDeleted: Retain    # or Delete
    whenScaled: Retain     # or Delete
```

Key Takeaway: Deleting StatefulSet ≠ deleting PVCs | Recreating same-name StatefulSet reattaches data | 1.27+ supports automatic PVC deletion

04 Scaling & Resource Management

Q14 HPA Not Scaling

You configured HPA but Pods are not scaling even under high CPU load. Debug the issue.

Answer:

HPA (Horizontal Pod Autoscaler) requires metrics-server to read CPU/memory metrics. If metrics-server is not installed or not working, HPA cannot make scaling decisions.

Step 1 — Check HPA status: `kubectl describe hpa <name>`. Look for 'unable to fetch metrics' or 'metrics not yet available'.

Step 2 — Check metrics-server: `kubectl get pods -n kube-system | grep metrics-server`. Also run `kubectl top pods` — if this fails, metrics-server is broken.

Step 3 — Verify resource requests: HPA CPU scaling is relative to `requests.cpu`. If the Pod has no CPU request defined, HPA cannot calculate a percentage and will not scale.

Step 4 — Check minReplicas: HPA will not scale below `minReplicas`. If `current replicas == maxReplicas`, it cannot scale up further.

Step 5 — Cooldown periods: HPA has a default scale-down cooldown of 5 minutes. It won't scale down immediately after a spike.

```
kubectl describe hpa my-hpa          # Check conditions
kubectl top pods                      # Metrics-server working?
kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods
# HPA spec requires resource requests:
resources:
  requests:
    cpu: 200m    # REQUIRED for HPA CPU scaling
```

Key Takeaway: HPA requires metrics-server | CPU requests are MANDATORY for CPU-based HPA | `kubectl top` fails = metrics-server down

Q15 ResourceQuota Blocking Deployments

A developer cannot create Pods in a namespace. The error mentions exceeded quota. How do you handle this?

Answer:

ResourceQuotas limit the total resources (CPU, memory, object counts) that can be used in a namespace. When a quota is exceeded, new Pods are rejected by the API server.

Step 1 — Check the quota: `kubectl describe resourcequota -n <namespace>`. Compare Used vs Hard limits for cpu, memory, and pods.

Step 2 — Find resource hogs: `kubectl top pods -n <namespace>` — identify which Pods consume the most resources.

Step 3 — Fix options:

- Increase the ResourceQuota Hard limits (requires cluster-admin)
- Delete unused Pods, Jobs, or other objects in the namespace
- Reduce resource requests in existing Deployments
- Move workloads to another namespace with available quota

Note: If a namespace has a ResourceQuota for CPU/memory, ALL Pods in that namespace must have resource requests defined. Pods without requests will be rejected.

```
kubectl describe resourcequota -n dev-team
kubectl get resourcequota -n dev-team -o yaml
kubectl top pods -n dev-team --sort-by=cpu
# Edit quota (cluster-admin required):
kubectl edit resourcequota my-quota -n dev-team
```

Key Takeaway: Quota enforces limits on namespace totals | All pods need requests when quota is set | LimitRange sets per-pod defaults

Q16 Node Pressure Eviction

Pods are being evicted unexpectedly. You see Evicted status. What is happening and how do you prevent it?

Answer:

Node pressure eviction occurs when a node runs low on resources (memory, disk, inodes). The kubelet starts evicting Pods to reclaim resources, starting with BestEffort QoS class, then Burstable, then Guaranteed.

QoS Classes:

- Guaranteed: requests == limits for ALL containers. Never evicted unless node is critically low.
- Burstable: requests < limits OR only some containers have requests. Evicted under pressure.
- BestEffort: NO requests or limits defined. First to be evicted.

Prevention:

- Set equal requests and limits (Guaranteed QoS) for critical workloads.
- Set PodDisruptionBudget to protect minimum replicas.
- Add cluster autoscaler to add nodes before pressure builds.

- Monitor node memory with Prometheus node_memory_MemAvailable_bytes.

```
kubectl get events --field-selector reason=Evicted
kubectl describe node <node> | grep -A10 Conditions
# Set Guaranteed QoS:
resources:
  requests:
    memory: 256Mi
    cpu: 200m
  limits:
    memory: 256Mi # Same as requests = Guaranteed
    cpu: 200m
```

Key Takeaway: requests==limits → Guaranteed QoS → last to evict | BestEffort = first evicted | PDB protects minimum availability

Q17 Cluster Autoscaler Not Adding Nodes

HPA wants to schedule more Pods but the Cluster Autoscaler is not adding new nodes. Why?

Answer:

Cluster Autoscaler (CA) adds nodes only when Pods are Unschedulable (Pending due to insufficient resources). It will NOT scale if Pods are just slow or if there are other reasons for pending.

Reasons CA may not scale:

- Max node group size reached — CA will not exceed the configured maximum.
- Pod has nodeSelector or affinity that restricts it to specific node types.
- Pod has PodAntiAffinity preventing it from sharing nodes with existing Pods.
- CA is disabled or not deployed in the cluster.
- Pod requests are very small — CA only adds nodes if a single node can fit the pending Pod.

Check CA logs: `kubectl logs -n kube-system -l app=cluster-autoscaler | grep -i 'scale up'`. The logs explain exactly why a scale-up was or was not triggered.

```
kubectl get pods --field-selector=status.phase=Pending
kubectl describe pod <pending-pod> | grep Events -A10
kubectl logs -n kube-system -l app=cluster-autoscaler | tail -100
# Check node group annotations:
kubectl describe node <node> | grep autoscaling
```

Key Takeaway: CA only triggers on Unschedulable pods | Check CA logs — they explain every decision | Hard affinity rules block CA expansion

05 RBAC & Security

Q18 Forbidden Access Error

A developer gets a 403 Forbidden error when running `kubectl get pods`. How do you diagnose and fix RBAC?

Answer:

Forbidden errors mean the user is authenticated but not authorized. RBAC determines what authenticated users can do.

Step 1 — Check who they are: `kubectl auth whoami (1.28+)` or check the kubeconfig context user.

Step 2 — Test the permission: `kubectl auth can-i get pods --namespace=dev --as=developer@company.com`. This simulates the exact permission check.

Step 3 — Find existing bindings: `kubectl get rolebindings,clusterrolebindings -o wide | grep <username>`. See what roles they currently have.

Step 4 — Grant the permission: Create a RoleBinding to bind a Role (namespace-scoped) or ClusterRoleBinding to bind a ClusterRole (cluster-wide).

Use principle of least privilege — grant only what is needed. Prefer Role over ClusterRole, and namespace-scoped access over cluster-wide.

```
kubectl auth can-i get pods --as=dev-user -n production
kubectl auth can-i '*' '*' --as=dev-user           # Check admin
# Create a RoleBinding:
kubectl create rolebinding dev-read \
  --clusterrole=view \
  --user=dev-user \
  --namespace=development
```

Key Takeaway: `auth can-i` is your RBAC debugger | Role = namespace-scoped, ClusterRole = cluster-wide | Least privilege principle always

Q19 ServiceAccount Token in Pod

An application running in a Pod needs to call the Kubernetes API (e.g., list Pods). How do you set this up securely?

Answer:

Pods can authenticate to the Kubernetes API using a ServiceAccount token, which is automatically mounted at `/var/run/secrets/kubernetes.io/serviceaccount/token`.

Step 1 — Create a dedicated ServiceAccount: Never use the default ServiceAccount for applications — it may have unwanted permissions or be used by multiple apps.

Step 2 — Create Role + RoleBinding: Grant only the specific verbs and resources the application needs.

Step 3 — Assign to Pod: Set `spec.serviceAccountName` in the Pod/Deployment spec.

Security hardening: Set `automountServiceAccountToken: false` on ServiceAccounts or Pods that don't need API access. In Kubernetes 1.24+, tokens are time-bound and audience-bound (no long-lived tokens by default).

```
kubectl create serviceaccount my-app-sa -n default
# Create Role:
kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods
# Bind it:
```

```
kubectl create rolebinding my-app-pod-reader \
  --role=pod-reader \
  --serviceaccount=default:my-app-sa
# Use in Pod:
spec:
  serviceAccountName: my-app-sa
```

Key Takeaway: Never use default SA for apps | automountServiceAccountToken: false if not needed | 1.24+ tokens are short-lived by default

Q20 Pod Security Context

A security audit flags that Pods are running as root. How do you fix this?

Answer:

Running containers as root violates the principle of least privilege. If a container is compromised, an attacker gets root access. Kubernetes provides securityContext to enforce non-root execution.

Container-level fix: Set securityContext.runAsNonRoot: true and runAsUser: <non-zero UID>. Kubernetes will fail the Pod if the image tries to run as root.

Additional hardening:

- readOnlyRootFilesystem: true — prevents writing to container FS
- allowPrivilegeEscalation: false — prevents sudo/setuid binaries
- capabilities.drop: [ALL] — removes all Linux capabilities

PodSecurityAdmission (PSA): Kubernetes 1.25+ uses Pod Security Standards (restricted, baseline, privileged) applied at namespace level. Set labels on namespace to enforce policies.

```
# Pod security context:
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
  containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: [ALL]
```

Key Takeaway: runAsNonRoot: true enforces non-root | readOnlyRootFilesystem blocks writes | PSA replaces PodSecurityPolicy since 1.25

Q21 Secret Management

A developer stores database passwords directly in environment variables in Deployment YAML committed to Git. What is the correct approach?

Answer:

Hardcoding secrets in YAML committed to Git is a critical security vulnerability. Kubernetes Secrets provide a better baseline, but for production use external secret management.

Level 1 — Kubernetes Secrets (minimum): Store secrets in a Secret object and reference via `envFrom` or `secretKeyRef`. Never store the raw value in YAML.

Level 2 — External Secrets (recommended): Use External Secrets Operator (ESO) to sync secrets from AWS Secrets Manager, HashiCorp Vault, or GCP Secret Manager into Kubernetes Secrets automatically.

Level 3 — Sealed Secrets: Use Bitnami Sealed Secrets — encrypt secrets with the cluster's public key so encrypted YAML can be safely committed to Git. Only the cluster can decrypt.

Rotate compromised secrets: If a secret was committed to Git: rotate the credential immediately, remove from Git history with `git filter-branch` or BFG Repo Cleaner, and audit access logs.

```
# Create secret (DO NOT commit the value to Git):
kubectl create secret generic db-creds \
  --from-literal=password=mypassword
# Reference in Pod:
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-creds
      key: password
```

Key Takeaway: Never commit raw secrets to Git | External Secrets Operator for production | Sealed Secrets enables GitOps with secrets

06 Observability & Debugging

Q22 Application Logs Not Available

kubectl logs <pod> returns 'error from server: context deadline exceeded'. What does this mean?

Answer:

This error means `kubectl` cannot reach the node where the Pod is running, or the kubelet on that node is not responding. The issue is infrastructure-level, not application-level.

Step 1 — Check the node: `kubectl get nodes`. If the node shows `NotReady`, the kubelet is unhealthy or the node is unreachable.

Step 2 — Check node conditions: `kubectl describe node <name>` shows conditions like `MemoryPressure`, `DiskPressure`, `NetworkUnavailable`.

Step 3 — SSH to the node: If you have access, SSH to the node and run `systemctl status kubelet` and `journalctl -u kubelet -f`.

Step 4 — Use centralized logging: In production, never rely solely on `kubectl logs`. Ship logs to a centralized system (ELK Stack, Loki+Grafana, Datadog). Logs survive Pod and node failures.

```
kubectl get nodes                # Check node status
kubectl describe node <node-name> # Node conditions
kubectl get events --sort-by=.lastTimestamp | tail -20
# Check kubelet on the node:
systemctl status kubelet
journalctl -u kubelet -f --since '5m ago'
```

Key Takeaway: context deadline exceeded = node unreachable | Always use centralized logging in production | Loki or ELK for log aggregation

Q23 High Restart Count

kubectl get pods shows RESTARTS column at 50+ for a critical service. How do you investigate?

Answer:

High restart count means the container is repeatedly crashing. Each restart increments the counter. The container could be crashing due to application errors, OOM, failing liveness probes, or misconfiguration.

Step 1 — Check previous logs: `kubectl logs <pod> --previous` — reads logs from the previous container instance.

Step 2 — Check exit code: `kubectl describe pod` shows Last State with the exit code. 137 = OOM, 1 = application error, 2 = misuse of shell command.

Step 3 — Check liveness probe: A misconfigured liveness probe (wrong path, too aggressive thresholds) will repeatedly kill a healthy container.

Step 4 — Check events: `kubectl get events --field-selector involvedObject.name=<pod-name>` shows a timeline of what happened.

Long-term fix: Set up alerting on `container_restarts > threshold` in Prometheus. Use Grafana to visualize restart trends over time.

```
kubectl logs <pod> --previous
kubectl describe pod <pod> | grep -A10 'Last State'
kubectl get events --field-selector involvedObject.name=<pod>
# Prometheus alert example:
sum(increase(kube_pod_container_status_restarts_total[1h])) > 5
```

Key Takeaway: --previous gets crashed container logs | Liveness probe misconfiguration = restart loop | Prometheus alert on high restarts

Q24 Prometheus Not Scraping Pod Metrics

A service is deployed but its metrics do not appear in Prometheus. What are the typical misconfigurations?

Answer:

Prometheus in Kubernetes (using kube-prometheus-stack or Prometheus Operator) discovers targets via service discovery. The most common issues are missing annotations or a missing ServiceMonitor.

If using annotations-based scraping: The Pod must have `prometheus.io/scrape: 'true'`, `prometheus.io/port: '8080'`, and `prometheus.io/path: '/metrics'` annotations.

If using ServiceMonitor (Prometheus Operator): The ServiceMonitor selector must match the Service labels. The Service must expose the metrics port.

Verify the target: In Prometheus UI, go to Status > Targets. Look for your service. If it's in DOWN or missing, the issue is discovery configuration.

Check Prometheus config: `kubectl describe prometheusrule / kubectl get servicemonitor -o yaml` to verify selectors match your Service.

```
# Pod annotations for annotation-based scraping:
metadata:
  annotations:
    prometheus.io/scrape: 'true'
    prometheus.io/port: '8080'
    prometheus.io/path: '/metrics'
# ServiceMonitor for Prometheus Operator:
spec:
  selector:
    matchLabels:
      app: my-service      # Must match Service labels
  endpoints:
    - port: metrics
      path: /metrics
```

Key Takeaway: Annotations-based vs ServiceMonitor — know which your cluster uses | Prometheus Targets page shows all discovery results | Labels must match exactly

Q25 Debugging Without `kubectl exec`

A container image has no shell (distroless). How do you debug a running Pod?

Answer:

Distroless images have no shell, no package manager, and no debug tools. `kubectl exec` won't work. Use these approaches instead.

Option 1 — Ephemeral Debug Containers (1.23+): `kubectl debug -it <pod> --image=busybox:latest --target=<container>`. This injects a temporary debug container into the running Pod's namespace.

Option 2 — Copy with debug: `kubectl debug <pod> -it --copy-to=debug-pod --image=ubuntu`. Creates a copy of the Pod with a debug image.

Option 3 — Node-level debugging: `kubectl debug node/<node-name> -it --image=ubuntu`. Runs a privileged Pod on the node for node-level debugging.

Option 4 — Port-forward + curl: `kubectl port-forward <pod> 8080:8080` and then test with `curl` from your local machine.

```
# Ephemeral debug container:
kubectl debug -it my-pod \
  --image=busybox:latest \
  --target=my-container
# Debug copy of pod:
```

```
kubectl debug my-pod -it \
  --copy-to=my-pod-debug \
  --image=ubuntu:22.04
# Port-forward for HTTP debugging:
kubectl port-forward pod/my-pod 8080:8080
```

Key Takeaway: Ephemeral containers = debug without modifying Pod | kubectl debug is the go-to for distroless | Port-forward bypasses Service/Ingress

07 Deployments & Configuration

Q26 ConfigMap Hot Reload

You updated a ConfigMap but the application is still using the old configuration. How does ConfigMap reload work?

Answer:

ConfigMap updates do NOT automatically restart Pods or reload application configs in real-time by default. The behavior depends on how the ConfigMap is consumed.

Mounted as Volume (file): Kubernetes updates mounted ConfigMap files automatically within ~1 minute (kubelet sync interval). However, the application must watch the file and reload — most apps don't do this by default.

Consumed as env vars (envFrom/valueFrom): Environment variables are set at Pod start. Updating the ConfigMap has NO effect until the Pod is restarted.

Force restart: `kubectl rollout restart deployment/<name>` — triggers a rolling restart to pick up new ConfigMap values.

Best practice — immutable ConfigMaps: Use a versioned ConfigMap name (app-config-v2) and update the Deployment reference. This creates a new rollout automatically and enables clean rollback.

```
# Force rolling restart to pick up ConfigMap changes:
kubectl rollout restart deployment/my-app
# Add restart annotation trigger (GitOps pattern):
kubectl patch deployment my-app -p \

'{"spec":{"template":{"metadata":{"annotations":{"kubectl.kubernetes.io/restartedAt":"2024-01-01"}}}}}'
# Immutable ConfigMap pattern:
# Name: app-config-v3 → update Deployment reference
```

Key Takeaway: Env vars don't reload — only restart helps | Volume mounts reload files but app must watch | Versioned ConfigMap names = clean rollback

Q27 Multi-Container Pod Communication

You have a sidecar container in the same Pod. How do they communicate, and what is the sidecar pattern used for?

Answer:

Containers in the same Pod share the same network namespace — they communicate via localhost with different ports. They also share volumes for file-based communication.

Network communication: If the main container listens on port 8080 and the sidecar on 9090, they reach each other via localhost:8080 and localhost:9090 respectively. No Service is needed.

File-based communication: Use a shared emptyDir volume. One container writes logs/data; the other reads and processes them.

Common sidecar patterns:

- Log shipper (Filebeat, Fluentd) — reads app logs from shared volume and ships to Elasticsearch
- Service mesh proxy (Envoy/Istio) — intercepts all network traffic for mTLS, observability
- Config watcher (Vault Agent) — fetches secrets and writes to shared volume
- Reverse proxy / TLS terminator (nginx) — handles HTTPS; app only deals with HTTP

```
spec:
  volumes:
    - name: shared-logs
      emptyDir: {}
  containers:
    - name: app
      ports: [{containerPort: 8080}]
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/app
    - name: log-shipper
      image: fluent/fluent-bit
      volumeMounts:
        - name: shared-logs
          mountPath: /var/log/app
```

Key Takeaway: Same Pod = same network namespace = localhost | emptyDir for shared files | Sidecar lifecycle tied to main container

Q28 Zero-Downtime Deployment

How do you configure a Deployment to achieve zero-downtime rolling updates?

Answer:

Zero-downtime deployments require the right combination of rolling update strategy, health probes, and preStop hooks.

1 — Rolling update strategy: Set strategy.type: RollingUpdate with maxUnavailable: 0 (no old Pods removed until new ones are Ready) and maxSurge: 1 (one extra Pod during rollout).

2 — Readiness probe: The new Pod must pass its readiness probe before the old one is terminated. Without this, traffic is sent to an unready Pod.

3 — preStop hook: Add a sleep in preStop to allow the load balancer time to deregister the Pod from its pool before SIGTERM is sent.

4 — minReadySeconds: Set minReadySeconds: 10 — Kubernetes waits this many seconds after a Pod becomes Ready before proceeding with the next replica.

5 — PodDisruptionBudget: Add a PDB with minAvailable: 1 to prevent node drains from taking all replicas offline simultaneously.

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  minReadySeconds: 10
  template:
    spec:
      containers:
        - lifecycle:
            preStop:
              exec:
                command: ["/bin/sh", "-c", "sleep 10"]
          readinessProbe:
            httpGet: {path: /ready, port: 8080}
```

Key Takeaway: maxUnavailable:0 = never remove old pod until new is ready | preStop sleep accounts for LB propagation delay | PDB prevents voluntary disruptions

Q29 Environment-Specific Config

You need to deploy the same application to dev, staging, and production with different configurations. What is the best approach?

Answer:

The best approach follows GitOps principles with environment-specific overlays, keeping the base config DRY and environment differences minimal.

Option 1 — Kustomize (built into kubectl): Define a base/ directory with common resources. Create overlays/dev, overlays/staging, overlays/prod that patch only the differences (replicas, image tag, resource limits, ConfigMap values).

Option 2 — Helm: A single Chart with values.yaml for defaults. Create values-dev.yaml, values-staging.yaml, values-prod.yaml. Deploy with helm install -f values-prod.yaml.

Option 3 — Separate namespaces: Deploy all environments to the same cluster in separate namespaces using namespace-level ResourceQuotas and NetworkPolicies to isolate them.

For production-grade setups, combine Helm/Kustomize with ArgoCD or Flux. Each environment has its own Git branch or directory, and changes are applied automatically on merge.

```
# Kustomize structure:
base/
  deployment.yaml
  kustomization.yaml
```

```
overlays/  
  prod/  
    kustomization.yaml    # patches replicas=5, image=v1.2.0  
  dev/  
    kustomization.yaml    # patches replicas=1, image=latest  
  
kubectl apply -k overlays/prod/
```

Key Takeaway: Kustomize is built-in (kubectl -k) | Helm for complex templating | ArgoCD/Flux for GitOps automation

08 StatefulSets & Databases

Q30 StatefulSet Ordered Scaling

A StatefulSet is scaling down and Pods are deleted in unexpected order. How does StatefulSet ordering work?

Answer:

StatefulSets maintain strict ordering and stable identity. Pods are created in order (0, 1, 2...) and deleted in reverse order (2, 1, 0). This ensures dependencies between replicas are respected.

Scale-up: Pod-0 must be Running and Ready before Pod-1 starts. Pod-1 before Pod-2, etc.

Scale-down: Pod with the highest ordinal is deleted first. Pod-2 before Pod-1, etc.

Parallel policy: Use podManagementPolicy: Parallel to start/stop all Pods simultaneously (like a Deployment). Useful when ordering doesn't matter (e.g., stateless apps that need stable network identity).

Updates: StatefulSet rolling updates also follow ordinal order in reverse — highest ordinal updated first, giving you time to verify before updating primary/leader.

```
spec:  
  replicas: 3  
  podManagementPolicy: OrderedReady # default  
  # or:  
  podManagementPolicy: Parallel     # all at once  
  updateStrategy:  
    type: RollingUpdate  
    rollingUpdate:  
      partition: 1 # Only update pods >= 1 (canary pattern)
```

Key Takeaway: Ordered scale-up protects leader election | partition: N enables canary updates for StatefulSets | Parallel policy sacrifices ordering for speed

Q31 Database Pod Affinity

You need to ensure database Pods are spread across different availability zones. How do you configure this?

Answer:

Use topologySpreadConstraints (preferred for Kubernetes 1.19+) or podAntiAffinity to spread Pods across availability zones or nodes.

topologySpreadConstraints: More flexible and declarative than anti-affinity. Specify maxSkew (max difference in Pods between zones), topologyKey (usually topology.kubernetes.io/zone), and whenUnsatisfiable policy.

podAntiAffinity (older approach): Use requiredDuringSchedulingIgnoredDuringExecution to hard-reject co-scheduling, or preferredDuringScheduling for best-effort.

Verify nodes have zone labels: `kubectl get nodes --show-labels | grep zone`. Cloud providers add topology.kubernetes.io/zone automatically.

```
# TopologySpreadConstraints (recommended):
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app: postgres
# Verify zone labels on nodes:
kubectl get nodes -L topology.kubernetes.io/zone
```

Key Takeaway: topologySpreadConstraints is preferred over anti-affinity | maxSkew:1 = maximum 1 Pod difference between zones | Cloud providers auto-label nodes with zone

09 Cluster Operations & Maintenance

Q32 Node Drain and Cordon

You need to perform maintenance on a node. What is the correct procedure to safely evacuate it?

Answer:

The correct sequence is: cordon the node (stop new Pods from scheduling), then drain it (evict existing Pods), then perform maintenance, then uncordon it.

kubectl cordon: Marks the node as unschedulable. No new Pods will be placed here. Existing Pods continue running.

kubectl drain: Evicts all non-DaemonSet, non-mirror Pods from the node. Respects PodDisruptionBudgets. Flags: `--ignore-daemonsets` (required), `--delete-emptydir-data` (for Pods using emptyDir).

After maintenance: `kubectl uncordon <node>` to make it schedulable again. Pods won't automatically move back (they're already running elsewhere), but new Pods can be scheduled here.

If drain hangs, it could be because a PDB is blocking eviction (minAvailable would be violated). You can use `--disable-eviction` or `--force` with `--grace-period=0` in emergencies (data loss risk).

```
kubectl cordon node-1 # Mark unschedulable
kubectl drain node-1 \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --grace-period=30
```

```
# Perform maintenance...
kubectl uncordon node-1           # Re-enable scheduling
kubectl get node node-1           # Verify Ready,SchedulingEnabled
```

Key Takeaway: cordon → drain → maintenance → uncordon | PDB can block drain — check if intentional | --force bypasses PDB (use only in emergencies)

Q33 etcd Backup

How do you take a backup of etcd in a kubeadm cluster, and why is it critical?

Answer:

etcd is the brain of Kubernetes — it stores ALL cluster state (all objects, configurations, secrets). If etcd data is lost without a backup, the entire cluster configuration is gone.

Take a snapshot: Use etcdctl snapshot save with the correct etcd certificates. In kubeadm clusters, the certs are at /etc/kubernetes/pki/etcd/.

Verify the snapshot: etcdctl snapshot status to confirm the backup is valid before storing it.

Storage: Store snapshots off-cluster (S3, GCS, NFS). Back up before every cluster upgrade and on a scheduled cron (e.g., every 6 hours in production).

Restore: etcdctl snapshot restore creates a new data directory. Update the etcd pod manifest to point to the new directory and restart etcd.

```
ETCDCTL_API=3 etcdctl snapshot save backup.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key

ETCDCTL_API=3 etcdctl snapshot status backup.db

# Restore:
ETCDCTL_API=3 etcdctl snapshot restore backup.db \
--data-dir=/var/lib/etcd-backup
```

Key Takeaway: etcd = entire cluster state | Backup before every upgrade | Store snapshots off-cluster always

Q34 Certificate Expiry

A kubeadm cluster's API server is unreachable. You find the TLS certificates have expired. How do you renew them?

Answer:

kubeadm generates certificates valid for 1 year by default. Expired certificates cause all control plane components to fail with TLS handshake errors.

Check expiry: kubeadm certs check-expiration shows all certificate expiry dates.

Renew all certificates: kubeadm certs renew all — renews all certificates that are close to or past expiry. Run on the control plane node.

Restart control plane: After renewal, the static Pod manifests are unchanged but the certs on disk are updated. Restart kube-apiserver, kube-controller-manager, kube-scheduler Pods by moving their manifests out of /etc/kubernetes/manifests and back.

Prevention: kubeadm upgrade automatically renews certificates. Set up a cron job to run kubeadm certs renew all every 6-11 months. Monitor expiry with Prometheus `ssl_certificate_expiry_seconds`.

```
kubeadm certs check-expiration          # View all cert expiry
kubeadm certs renew all                 # Renew all certs
# Restart static pods after renewal:
mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
sleep 5
mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/
# Update kubeconfig after renewal:
cp /etc/kubernetes/admin.conf ~/.kube/config
```

Key Takeaway: kubeadm certs expire in 1 year | Upgrading cluster also renews certs | Alert at 30 days before expiry

Q35 Node NotReady

A node shows NotReady status. What is your investigation and recovery process?

Answer:

NotReady means the kubelet on that node has not sent a heartbeat to the API server for the node timeout period (default: 40 seconds). The node could be unreachable, or the kubelet could be down.

Step 1 — Describe the node: `kubectl describe node <name>` shows Conditions (MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable, Ready).

Step 2 — Check node resources: SSH to the node if possible. Run `free -h` (memory), `df -h` (disk), `top` (CPU). High resource pressure can cause kubelet to become unresponsive.

Step 3 — Check kubelet: `systemctl status kubelet`. `journalctl -u kubelet -f` for logs. Common issues: disk full blocking kubelet, container runtime crashed, certificate expired.

Step 4 — Check container runtime: `systemctl status containerd` or `docker`. If the runtime is down, kubelet cannot manage containers.

Recovery: Fix the root cause, restart kubelet (`systemctl restart kubelet`), and verify the node transitions back to Ready within ~1 minute.

```
kubectl describe node <node>          # Read Conditions
kubectl get events --field-selector involvedObject.name=<node>
# On the node itself:
systemctl status kubelet
journalctl -u kubelet --since '10m ago'
systemctl status containerd
df -h /var/lib/kubelet                 # Disk pressure?
```

Key Takeaway: NotReady = kubelet heartbeat lost | Check disk/memory/runtime on the node | `systemctl restart kubelet` usually fixes transient issues

10 Advanced & Real-World Scenarios

Q36 Canary Deployment

You want to release a new version to only 10% of users before full rollout. How do you implement a canary deployment in Kubernetes?

Answer:

Kubernetes doesn't have native canary support, but you can implement it using replica weighting or an Ingress controller with traffic splitting.

Option 1 — Replica weighting: Run 9 replicas of v1 and 1 replica of v2. Traffic is distributed proportionally (10% to v2). Both Deployments have the same Service selector label (e.g., app: my-app).

Option 2 — NGINX Ingress canary: Use `nginx.ingress.kubernetes.io/canary: 'true'` and `nginx.ingress.kubernetes.io/canary-weight: '10'` annotations on a second Ingress pointing to the v2 Service.

Option 3 — Argo Rollouts: The proper solution for production. Argo Rollouts provides CRD-based Canary and Blue/Green strategies with automatic analysis, metrics-based promotion, and rollback.

```
# NGINX Ingress canary:
metadata:
  annotations:
    nginx.ingress.kubernetes.io/canary: 'true'
    nginx.ingress.kubernetes.io/canary-weight: '10'
# Argo Rollouts canary spec:
strategy:
  canary:
    steps:
      - setWeight: 10
      - pause: {duration: 10m}
      - setWeight: 50
      - pause: {duration: 10m}
```

Key Takeaway: Replica weighting is simple but coarse-grained | NGINX canary for header/cookie-based routing | Argo Rollouts for production-grade canary

Q37 Blue/Green Deployment

How do you implement a blue/green deployment to achieve instant cutover with zero downtime?

Answer:

Blue/green runs two identical environments (blue=current, green=new). Traffic is switched from blue to green all at once by updating a Service selector. Rollback is instant — just switch back.

Step 1 — Deploy green: Create green Deployment with version: green label. Both blue and green run simultaneously. The Service still points only to blue.

Step 2 — Test green: Access green directly via a temporary Service or port-forward. Run smoke tests.

Step 3 — Cutover: Update the main Service selector from version: blue to version: green. All traffic switches instantly.

Step 4 — Cleanup: After verifying green is stable (15-30 minutes), delete the blue Deployment.

Note: Blue/green requires 2x resources during the transition. For resource-constrained clusters, use a rolling update instead.

```
# Service pointing to blue:
spec:
  selector:
    app: my-app
    version: blue      # ← change to 'green' for cutover

# Cutover command:
kubectl patch service my-app -p \
  '{"spec":{"selector":{"version":"green"}}}'

# Instant rollback:
kubectl patch service my-app -p \
  '{"spec":{"selector":{"version":"blue"}}}'
```

Key Takeaway: Cutover = single kubectl patch = instant switch | Rollback = patch back to blue = zero downtime | Needs 2x resources during transition

Q38 Job and CronJob Failures

A CronJob is not running on schedule and some Job Pods are failing. How do you troubleshoot?

Answer:

CronJob failures are often caused by missed schedules, concurrency policy conflicts, or the Job's Pods themselves failing.

Check CronJob status: `kubectl describe cronjob <name>`. Look at Last Schedule, Active Jobs, and Events. Missed schedules (startingDeadlineSeconds exceeded) are logged here.

Check Job history: `kubectl get jobs -l job-name=<cronjob-name>`. CronJobs keep successfulJobsHistoryLimit (default 3) and failedJobsHistoryLimit (default 1) Jobs.

Common issues:

- startingDeadlineSeconds too short: If the controller missed the window (due to cluster downtime), it won't reschedule.
- concurrencyPolicy: Forbid: If the previous Job is still running, the new one won't start.
- Job Pods failing: `kubectl logs <job-pod>` for the actual error. Check backoffLimit (default 6 retries).
- Clock skew: Ensure cluster nodes have synchronized clocks (NTP). CronJob schedule is based on controller-manager time.

```
kubectl describe cronjob my-cron      # Last schedule info
kubectl get jobs --sort-by=.metadata.creationTimestamp
kubectl logs -l job-name=my-cron-job  # Latest job logs
```

```
# Manually trigger a CronJob for testing:
kubectl create job --from=cronjob/my-cron manual-run-1
```

Key Takeaway: Manually trigger with `kubectl create job --from=cronjob` | `concurrencyPolicy: Forbid` blocks overlapping jobs | Check controller-manager clock for schedule issues

Q39 Helm Upgrade Failing

A helm upgrade is failing and left the release in a failed state. How do you recover?

Answer:

A failed Helm upgrade leaves the release in a failed state, blocking future upgrades. You must either rollback or fix the release state.

Step 1 — Check release status: `helm list -a` shows releases including failed ones. `helm history <release>` shows all revisions.

Step 2 — Rollback: `helm rollback <release> <revision>`. This reinstates the previous working revision. Use `helm history` to find the last successful revision.

Step 3 — If rollback fails: Use `helm uninstall <release>` and redeploy from scratch. Or manually patch the release secret: `kubectl get secret sh.helm.release.v1.<release>.v<n>` to reset state.

Prevention: Use `--atomic` flag in `helm upgrade`. If the upgrade fails, it automatically rolls back to the previous revision. Use `--timeout` to define how long to wait before considering an upgrade failed.

```
helm list -a                # Show all releases
helm history my-release     # Show all revisions
helm rollback my-release 3  # Rollback to revision 3

# Safe upgrade pattern:
helm upgrade my-release ./chart \
  --atomic \
  --timeout 5m \
  --wait

helm uninstall my-release   # Nuclear option
```

Key Takeaway: `--atomic` auto-rolls back on failure | `helm history` shows all revisions | `FAILED` state blocks upgrades until rollback

Q40 Multi-Tenancy Namespace Isolation

Your cluster serves multiple teams. How do you implement proper isolation between namespaces?

Answer:

Multi-tenancy in Kubernetes requires layered isolation: resource boundaries, network isolation, and access control.

1 — Resource isolation (ResourceQuota + LimitRange): `ResourceQuota` limits total CPU/memory/object counts per namespace. `LimitRange` sets default requests/limits for containers that don't specify them.

2 — Network isolation (NetworkPolicy): By default, all namespaces can communicate. Apply a default-deny NetworkPolicy in each namespace, then explicitly allow required cross-namespace traffic.

3 — RBAC isolation: Each team gets a ServiceAccount and RoleBinding scoped to their namespace. Use ClusterRole with namespace-scoped RoleBinding, not ClusterRoleBinding.

4 — Pod Security Standards: Apply PSA at namespace level: `kubectl label namespace team-a pod-security.kubernetes.io/enforce=restricted`.

Hard vs Soft multi-tenancy: For true hard isolation (hostile tenants), use separate clusters or Kubernetes virtual clusters (vCluster). Namespace isolation is 'soft' — a compromised container could potentially escape.

```
# Default deny NetworkPolicy:
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
spec:
  podSelector: {}          # Applies to all pods
  policyTypes:
    - Ingress
    - Egress
# PSA enforce label:
kubectl label namespace team-a \
  pod-security.kubernetes.io/enforce=restricted
```

Key Takeaway: Namespaces = soft isolation | Separate clusters for hard multi-tenancy | Combine ResourceQuota + NetworkPolicy + RBAC + PSA

Q41 Sidecar Injection Failing

Istio sidecar injection is not working for Pods in a namespace. How do you debug this?

Answer:

Istio sidecar injection works via a MutatingWebhookConfiguration that intercepts Pod creation and injects the Envoy proxy. If it's not working, the webhook is not being called or is misconfigured.

Step 1 — Check namespace label: `kubectl get namespace <name> --show-labels`. Must have `istio-injection: enabled` label.

Step 2 — Check webhook: `kubectl get mutatingwebhookconfiguration istio-sidecar-injector`. Verify it's present and the namespaceSelector matches.

Step 3 — Check istiod: `kubectl get pods -n istio-system`. `istiod` must be Running. Unhealthy `istiod` = no injection.

Step 4 — Pod annotation override: Individual Pods can opt out with `sidecar.istio.io/inject: 'false'` annotation. Check if this is set.

Step 5 — Restart Pods: Injection only happens at Pod creation. Existing Pods need to be restarted: `kubectl rollout restart deployment/<name>`.

```
kubectl label namespace my-ns istio-injection=enabled
kubectl get mutatingwebhookconfiguration | grep istio
kubectl get pods -n istio-system
```

```
# Check if pod has sidecar:
kubectl get pod <pod> -o jsonpath='{.spec.containers[*].name}'
# Force restart for injection:
kubectl rollout restart deployment -n my-ns
```

Key Takeaway: Label namespace THEN restart pods | Injection is per-Pod-creation, not retroactive |
sidecar.istio.io/inject: false to exclude specific pods

Q42 Pod Disruption Budget

During a node drain, all replicas of a critical service were evicted simultaneously causing an outage. How do you prevent this?

Answer:

A PodDisruptionBudget (PDB) limits the number of Pods that can be voluntarily disrupted simultaneously. It protects against simultaneous eviction during node drains, cluster upgrades, and node auto-scaling.

minAvailable: Specifies the minimum number of Pods that must be available. E.g., minAvailable: 2 means at most (replicas - 2) Pods can be disrupted at once.

maxUnavailable: Specifies the max number of Pods that can be unavailable. E.g., maxUnavailable: 1 means at most 1 Pod can be down at any time.

Limitations: PDB only affects VOLUNTARY disruptions (drain, eviction). It does NOT protect against node failures (involuntary disruptions). For involuntary protection, use multiple replicas across different nodes/zones.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 2      # or: maxUnavailable: 1
  selector:
    matchLabels:
      app: my-app
# Verify PDB status:
kubectl get pdb
kubectl describe pdb my-app-pdb
```

Key Takeaway: PDB protects voluntary disruptions only | minAvailable as integer or percentage (50%) | PDB blocks node drain if violation would occur

Q43 ImagePullBackOff

Pods are in ImagePullBackOff. Walk through all possible causes and their fixes.

Answer:

ImagePullBackOff means the kubelet could not pull the container image. It retries with exponential backoff. The causes range from typos to authentication failures.

Cause 1 — Wrong image name or tag: The most common cause. Verify: kubectl describe pod shows the exact image being pulled. Check for typos, wrong registry URL, or a tag that doesn't exist.

Cause 2 — Private registry auth: The registry requires authentication. Create a docker-registry Secret and reference it in imagePullSecrets in the Pod spec or ServiceAccount.

Cause 3 — Rate limiting: Docker Hub limits unauthenticated pulls (100/6h for IPs). Authenticate to increase limits or use a registry mirror/cache.

Cause 4 — Node network issue: The node cannot reach the registry. SSH to the node and run docker pull <image> manually to test.

Cause 5 — imagePullPolicy: imagePullPolicy: Always forces a pull every time. If the registry is temporarily down, a running Pod will fail to restart. Use IfNotPresent for stability.

```
kubectl describe pod <pod> | grep -A5 Events
# Create registry secret:
kubectl create secret docker-registry regcred \
  --docker-server=registry.example.com \
  --docker-username=user \
  --docker-password=pass
# Reference in Pod:
spec:
  imagePullSecrets:
    - name: regcred
  containers:
    - image: registry.example.com/my-app:v1.0
```

Key Takeaway: Describe pod Events shows exact pull error | imagePullSecrets per-Pod or on the ServiceAccount | imagePullPolicy: IfNotPresent for stability

Q44 Kubernetes Upgrade Strategy

You need to upgrade a production kubeadm cluster from 1.28 to 1.30. What is the procedure and what can go wrong?

Answer:

Kubernetes supports upgrading one minor version at a time. 1.28 → 1.29 → 1.30. Skipping minor versions is not supported.

Phase 1 — Control plane upgrade: Upgrade kubeadm, then run kubeadm upgrade apply v1.29.x. This upgrades the API server, controller manager, and scheduler. Then upgrade kubectl and kubelet on the control plane node.

Phase 2 — Worker node upgrade: Drain each worker node, upgrade kubeadm/kubelet/kubectl on the node, run kubeadm upgrade node, then uncordon. Do one node at a time in production.

Pre-upgrade checklist:

- Take an etcd backup
- Check for deprecated APIs (kubectl api-versions, Pluto tool)
- Test the upgrade in staging first
- Check addon compatibility (CoreDNS, kube-proxy, CNI plugin)

What can go wrong: API deprecations breaking existing YAML, addon version incompatibility, PDB blocking node drains, etcd issues during control plane upgrade.

```
# Control plane node:
apt update && apt install -y kubeadm=1.29.x-*
kubeadm upgrade plan
kubeadm upgrade apply v1.29.0
apt install -y kubelet=1.29.x-* kubectl=1.29.x-*
systemctl restart kubelet

# Worker nodes (repeat for each):
kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data
# On the node: upgrade kubeadm/kubelet/kubectl
kubeadm upgrade node
systemctl restart kubelet
kubectl uncordon node-1
```

Key Takeaway: One minor version at a time | etcd backup before every upgrade | Pluto tool for deprecated API detection

Q45 Persistent Volume Expansion

A developer needs more storage on an existing PVC. How do you expand it without data loss?

Answer:

PVC expansion is possible if the StorageClass has `allowVolumeExpansion: true`. The procedure differs depending on whether the volume type supports online or offline expansion.

Step 1 — Check StorageClass: `kubectl get storageclass <name> -o yaml | grep allowVolumeExpansion`. Must be true.

Step 2 — Edit the PVC: `kubectl edit pvc <name>` and increase `spec.resources.requests.storage`. Kubernetes will expand the underlying volume.

Step 3 — For filesystem resize: If the volume needs a filesystem resize (EBS, GCE PD), Kubernetes automatically triggers it when the Pod mounts the volume. No restart required for online-expansion-capable drivers.

Step 4 — Verify: `kubectl get pvc <name> — status.capacity` should reflect the new size. `df -h` inside the Pod confirms the filesystem is expanded.

Note: You can only expand a PVC, never shrink it. Shrinking is not supported and requires creating a new PVC and migrating data.

```
kubectl get storageclass standard -o yaml | grep allowVolumeExpansion
kubectl edit pvc my-pvc
# In the editor, change:
# spec.resources.requests.storage: 10Gi → 20Gi

# Monitor the resize:
kubectl describe pvc my-pvc | grep -A5 Conditions
kubectl get pvc my-pvc -o jsonpath='{.status.capacity}'
# Inside pod:
```

```
df -h /data
```

Key Takeaway: allowVolumeExpansion must be true on StorageClass | PVC can only grow, never shrink | No Pod restart needed for online-expansion CSI drivers

Q46 Liveness vs Readiness Probe Confusion

A senior engineer says your liveness probe configuration will cause unnecessary restarts during high load. What mistake did you likely make?

Answer:

A common mistake is making the liveness probe too similar to the readiness probe, or setting thresholds too aggressively — causing containers to be killed during temporary slowdowns.

The mistake: Using the same endpoint and thresholds for both liveness and readiness. Under high CPU/memory load, the app might be slow to respond. A tight liveness probe (short timeout, low failureThreshold) kills the container, making the situation worse (restart + warm-up time).

Correct philosophy:

- Readiness probe: Frequent checks (every 5s), lower thresholds. Temporarily removes Pod from LB during slowdowns.
- Liveness probe: Infrequent checks (every 30s), higher failureThreshold (3-5). Only kills the container if it's truly dead/deadlocked.
- Startup probe: For slow starters — use high failureThreshold * periodSeconds to give max startup time.

Rule: Liveness = is the process alive? Readiness = is the process ready for traffic? Never use identical configs for both.

```
# WRONG: same aggressive config for both
livenessProbe:
  httpGet: {path: /health, port: 8080}
  periodSeconds: 5
  failureThreshold: 1
# RIGHT: differentiated thresholds
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 30      # Infrequent
  failureThreshold: 5    # Patient
  timeoutSeconds: 5
readinessProbe:
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5      # Frequent
  failureThreshold: 3
```

Key Takeaway: Liveness = is process alive (patient) | Readiness = ready for traffic (frequent) | Never use identical thresholds for both

Q47 Port-Forward vs Service

When should you use `kubectl port-forward` vs a Kubernetes Service to access a Pod?

Answer:

kubectl port-forward and Services serve completely different purposes and are used in different contexts.

kubectl port-forward:

- Purpose: Direct, ephemeral access to a specific Pod for debugging and development.
- How: Creates an SSH tunnel from your local machine to a specific Pod. Traffic bypasses all Services, Ingress, and NetworkPolicies.
- Use cases: Debug a specific Pod instance, access databases directly, test a new Pods behavior, access admin UIs (Prometheus, Grafana).
- Limitation: Ties to one Pod. If the Pod is deleted/restarted, the tunnel breaks.

Kubernetes Service:

- Purpose: Stable, production-grade network endpoint for one or more Pods.
- How: Load balances across all matching Pods, survives Pod restarts, provides DNS name.
- Use cases: All production traffic, inter-service communication, exposing to external users.

```
# port-forward: debug a specific pod
kubectl port-forward pod/my-app-xyz-abc 8080:8080
kubectl port-forward svc/my-service 8080:80    # Via service

# Service: stable production access
# ClusterIP: internal only
# NodePort: node IP + port
# LoadBalancer: cloud external IP
```

Key Takeaway: port-forward = debug tool, not production | Service = stable endpoint with load balancing | port-forward bypasses NetworkPolicy — useful for debugging